Documento final del proyecto

Proyecto de Especialización en Herramientas de Testing — Sprint 4: Especialización, Optimización y Defensa Final

- **Cliente:** QualityTech Solutions
- **Equipo:** Grupo 5, IES Alfredo Coviello. Cesar Augusto Basaury Marquesto, Germán Garnica, Jorge Alberto Medina y Leandro Raúl Vallejo.
- **Herramienta:** Postman + Newman, con GitHub Actions como pipeline.
- **Sistema bajo prueba:** API REST de una plataforma de cadetería (Java 21, Spring Boot 3.3.5, MySQL 8), versión `30a29d1`, desde el Sprint 2. En el Sprint 1 fue la API de Gestión de Alumnos del instituto.
- **Entrega:** 29/10/2026

---

1. Resumen

En ocho semanas pasamos de probar a mano 15 casos en Postman a una suite de 22 casos que corre sola, contra la API real, en cada cambio y todos los días, y que publica su reporte en una página.

El aprendizaje más importante del proyecto llegó en este sprint. **La suite del Sprint 3 daba 100 % de casos aprobados, pero contra un servidor simulado (mock).** Al correrla contra la API real con una base nueva, falló en las 30 corridas que hicimos. El mock no conocía una regla de negocio que el backend había incorporado: la versión mínima de la app. Además, un caso dependía de datos de ejemplo que se reparten al azar.

Este sprint no fue de hacer más, fue de hacer mejor:

	Sprint 3	Sprint 4
Contra qué corre el pipeline	Mock (servidor simulado)	API real (MySQL + backend dentro de GitHub Actions)
Corridas en verde con base nueva (3 bases × 10 corridas)	0 de 30	**30 de 30**
Casos de prueba	15	22 (15 estabilizados + 7 de reglas de negocio)
Código del backend ejercitado (líneas, JaCoCo)	27,6 %	32,2 %
Ramas (if/else) ejercitadas	5,4 %	9,7 %
Reglas de negocio verificadas	0	4

2. Evolución del proyecto (8 semanas)

Sprint	Semanas	Qué hicimos	Qué aprendimos
1	1–2	**Fundamentos y entorno.** Investigación teórica: testing de APIs, pirámide de testing, Postman (colecciones, entornos, scripts), Newman y OWASP API Security Top 10. Elegimos Postman. El primer sistema bajo prueba fue la API de **Gestión de Alumnos** del instituto, un proyecto propio en Spring Boot y MySQL que levantamos en local.	La primera evidencia ya fue un hallazgo: con una contraseña equivocada, el login de esa API respondía **500** (error del servidor) en lugar de 401. Un error del cliente no tiene que verse como una caída del servidor.

Sprint	Semanas	Qué hicimos	Qué aprendimos
—	—	**Cambio de sistema bajo prueba.** Al empezar el Sprint 2 pasamos a la API de cadetería por dos motivos: es un sistema con un cliente real que lo usa, con reglas de negocio reales para probar, y le podíamos dedicar más tiempo. La probamos siempre sobre una copia propia (la del instituto, o una levantada en local o en el pipeline), nunca sobre la del cliente.	Elegir el sistema bajo prueba también es una decisión de testing: conviene uno con reglas de negocio para probar y un formato de errores estable.
2	3–4	**Diseño de pruebas y primeras automatizaciones.** 15 casos sobre 5 endpoints (1 positivo + 2 negativos por endpoint), matriz de trazabilidad REQ → CP, técnicas de partición de equivalencia, valores límite, transición de estados y seguridad por roles. Encadenamiento de tokens e ids con pasos de preparación (P1–P3). 100 % de aprobados en Postman.	La sesión única por cadete obliga a ordenar los casos (el token "vencido" se obtiene antes del login del caso 01). En las lecciones aprendidas escribimos: *"incorporaríamos desde el inicio Newman en un pipeline de CI/CD"*.
3	5–6	**Automatización y CI.** Newman con reportes HTML y JSON, entornos local y CI, chequeos globales (tiempo de respuesta, JSON, formato de error), paso de limpieza Z1, mock del contrato y GitHub Actions. Aserciones por corrida: de 35 a 90.	Cumplimos lo que habíamos dicho en el Sprint 2. Pero el pipeline solo corría contra el mock: el job contra la API real quedó configurado y nunca se ejecutó.
4	7–8	**Optimización y defensa.** Corrimos la suite contra la API real y encontramos que no era estable. La estabilizamos, reemplazamos el mock por la API real dentro del pipeline, sumamos un flujo corto de reglas de negocio, cobertura de código, publicación del reporte y métricas reales.	Un verde que no prueba contra el sistema real es una falsa seguridad. Una suite vale por su confiabilidad, no por su cantidad de casos.

3. Estrategia de prueba

Prueba basada en riesgo. La API tiene 201 endpoints en 38 controladores. No buscamos cubrirlos todos: elegimos los que, si fallan, cortan el trabajo de un cadete o cuestan plata:

1. **Acceso** (login, sesión única, permisos por rol): si falla, un cadete no puede trabajar o alguien ve lo que no debe.
2. **Disponibilidad del cadete** (Libre / Ocupado / Desconectado y versión mínima de la app): si falla, se ofrecen viajes a cadetes que no pueden tomarlos.
3. **Ciclo de vida de un pedido** (alta → asignación → aceptación → retiro → entrega): es el producto que se vende.
4. **Controles antifraude** (espera mínima entre retiro y entrega, cierre manual con motivo): si fallan, se paga un viaje que no se hizo.

Técnicas aplicadas: - Partición de equivalencia: estado válido o inexistente, rol correcto o equivocado. - Valores límite: tipo de dato en `versionApp`, versión de app 0 contra la vigente. - Transición de estados: `SIN_ASIGNAR → EN_CURSO → FINALIZADO`, y Libre/Desconectado del cadete. - Pruebas negativas de seguridad: token mal formado, vencido o ausente, y rol cruzado.

Niveles de verificación por request: - Del caso: código HTTP, `codigo` de error y el dato de negocio (estado del pedido, hora de retiro, mensaje). - Globales: tiempo de respuesta, `Content-Type` y formato de error, escritos una sola vez a nivel de colección.

4. Decisiones técnicas del Sprint 4 (y por qué)

#	Decisión	Por qué	Alternativa descartada
1	**El pipeline corre contra la API real**, levantada dentro de GitHub Actions: un contenedor MySQL 8 nuevo y el backend `30a29d1` compilado desde su repositorio	Es la única forma de que el verde signifique algo. La base nace vacía con la carga de demostración en cada ejecución: no hay estado heredado.	Seguir con el mock: daba verde donde la API real fallaba (ver H1).
2	**Se elimina el mock**	Su contrato había quedado viejo y mantenerlo era deuda técnica. Con la API real en el pipeline ya no aporta nada.	Mantener los dos: doble mantenimiento y dos resultados distintos para la misma suite.
3	**La "CD" es publicar el reporte, no desplegar**	El servidor del instituto es compartido con un sistema en producción con clientes reales. Un pipeline con permiso de desplegar ahí es un riesgo que no se justifica en un proyecto de testing. El pipeline no tiene ninguna clave del servidor.	Desplegar el backend si los tests pasan: el backend no es de este repositorio y pondría en riesgo producción.
4	**El job contra el instituto es manual y de una sola vuelta**, con las contraseñas en secretos de GitHub	Reproduce lo que el equipo hacía a mano y sirve para la demo, sin cargar el servidor con cada push. `concurrency` evita dos corridas a la vez.	Correrlo en cada push: tráfico automático contra un servidor compartido.
5	**No hay pruebas de carga contra el servidor**	El instituto y el sistema en producción comparten máquina (2 vCPU, 4 GB) y base de datos. Una prueba de carga contra uno degrada al otro.	Medir rendimiento en el servidor real. Los tiempos se miden en la API aislada.
6	**Los cambios de estado mandan `X-App-Version`**, como la APK real	Sin el header, la API usa la versión del último login, o 0 si nunca la mandó. El resultado dependía de quién había entrado antes (ver H2).	Confiar en la versión del último login: falla con base nueva.
7	**La suite crea sus propios datos**: el caso 13 usa el pedido que crea el flujo	La carga de demostración reparte pedidos e incidentes "al primer cadete que devuelve la base", que depende de un UUID al azar (ver H3).	Tomar un pedido del historial: falló en una de seis bases nuevas.
8	**Precondición explícita P6**: antes del flujo se cierran los incidentes abiertos del cadete, como lo haría el admin	Un cadete con un reclamo abierto no puede recibir pedidos, y la carga demo deja uno abierto a un cadete al azar.	Usar otro cadete: el problema se mueve, no se resuelve.
9	**El caso 16 manda la versión 0 a propósito**	Para que "app vieja" sea siempre vieja, sin depender de logins anteriores.	Omitir el header: el resultado varía según la historia de la base.
10	**Flujo corto (7 casos), no exhaustivo**	El sprint pide hacer mejor, no más. 7 casos alcanzan para cubrir el camino crítico y las reglas antifraude.	18 casos con todas las variantes: más mantenimiento para poco riesgo adicional.

#	Decisión	Por qué	Alternativa descartada
11	**Cobertura con el agente de JaCoCo pegado a la API mientras corre la suite**	Mide qué código del backend ejercita *esta* suite, sin tocar el proyecto del backend.	Correr los tests unitarios del backend: miden otra cosa.
12	**Corrida diaria programada**	Detecta si algo cambió (dependencias, imagen de MySQL) aunque nadie haya subido cambios.	Solo en cada push: un repo quieto nunca avisa.
13	**Precondición P1b**: si el cadete de prueba no tiene la cuota semanal paga, la suite la habilita como el admin	El backend deshabilita a todos los cadetes de pago semanal cada lunes: sin esto, el job contra el instituto fallaría todas las semanas (ver H8).	Habilitarlo a mano antes de cada corrida.
14	**Iteraciones configurables** (`-n`) desde el botón del workflow	Permiten medir la estabilidad (10 corridas seguidas) sin tocar el pipeline.	Repetir a mano.

5. Métricas

Mediciones en una PC con la misma configuración que el pipeline: MySQL 8, backend `30a29d1`, base nueva con la carga de demostración y JaCoCo 0.8.12. Las corridas del Sprint 3 usan su colección tal como se entregó.

Confiabilidad

Métrica	Sprint 3	Sprint 4
Corridas en verde contra el mock	100 %	— (mock eliminado)
Corridas en verde contra la API real, base nueva (3 bases × 10 corridas)	**0 / 30** (Z1 falla siempre)	**30 / 30**
Fallos sin un error real del sistema (falsos negativos)	Z1 siempre; caso 13 en 1 de 6 bases nuevas	0 en 30 corridas
Aserciones fallidas	30 de 2.730	0 de 3.875

Cobertura

Métrica	Sprint 3	Sprint 4
Casos de prueba	15	22
Endpoints con casos de prueba	5 de 201	11 de 201
Líneas del backend ejercitadas	27,6 % (2.439 / 8.844)	**32,2 %** (2.846 / 8.844)
Ramas (if/else) ejercitadas	5,4 % (190 / 3.507)	**9,7 %** (341 / 3.507)
Métodos ejercitados	27,8 %	32,5 %
Reglas de negocio verificadas	0	4 (versión mínima, solo cadete libre, espera mínima de entrega, cierre manual con motivo)

La cobertura de líneas parece baja, pero hay que leerla bien. La suite es de **API** y apunta al camino crítico: ejercita el 32 % del backend con 22 casos. Más de un tercio de ese número lo pone el arranque de la aplicación: el paquete `config` (configuración y carga de datos) suma 1.021 de esas 2.846 líneas. El salto en **ramas** (de 5,4 % a 9,7 %, casi el doble) es lo que muestra que ahora se prueban decisiones de negocio y no solo "responde 200".

Velocidad

Métrica	Valor
Una corrida completa de la suite (28 requests)	≈ 3,5 s
Tiempo de respuesta promedio / p95 de la API (local)	47 ms / 181 ms
Arranque del backend con base nueva	≈ 17 s
Pasos manuales para correr y obtener el reporte	0 (push al repositorio, o `npm test`)
Duración total del pipeline en GitHub Actions	≈ 1 min 40 s: 92 s el job de pruebas (30 s levantar MySQL, 14 s compilar el backend, 18 s arrancarlo, 3 s la suite) y 9 s publicar el reporte
Resultado en GitHub Actions	22 / 22 casos, 134 aserciones sin fallos, cobertura de líneas 32,4 %
Reporte publicado	https://augustoba.github.io/suite-api-cadeteria/

6. Hallazgos

El detalle, con el impacto para el negocio y la recomendación de cada uno, está en `analisis-resultados.md`.

#	Hallazgo	Tipo	Severidad
H1	El mock daba verde donde la API real falla	Proceso de prueba	Alta
H2	La versión de la app que se controla depende del último login si no viene el header	Comportamiento del sistema / prueba	Media
H3	Los datos de demostración se reparten al azar entre cadetes	Datos de prueba	Media
H4	El job contra la API real del Sprint 3 nunca podía funcionar	Pipeline	Media
H5	Cualquiera que conozca el DNI de un cadete puede bloquearle la cuenta 15 minutos	Seguridad	**Alta**
H6	Asignar un pedido anulado devuelve un mensaje que no corresponde	Usabilidad	Baja
H8	El entorno compartido no estaba en condiciones de probar (cuotas deshabilitadas cada lunes, contraseña temporal vencida)	Entorno de prueba	Media
H7	Las reglas antifraude y de disponibilidad funcionan como están especificadas	Verificación	—

7. Lecciones aprendidas

1. **Un verde vale lo que vale el sistema contra el que corre.** El mock nos dio tranquilidad durante un sprint entero, y era falsa.
2. **Un pipeline "configurado" no es un pipeline funcionando.** El job contra la API real tenía el nombre del archivo equivocado y nunca había corrido. Y nos volvió a pasar: nuestra primera corrida en GitHub falló porque `actions/checkout` no acepta el hash corto del commit, algo que en local no se veía. Solo se sabe si se lo ve en verde al menos una vez.
3. **La inestabilidad casi nunca es de la herramienta: es de los datos.** Los dos fallos que no eran del sistema venían de depender de datos que la suite no controlaba. La solución fue que la suite cree o prepare lo que necesita.
4. **Correr la suite contra un entorno compartido tiene consecuencias.** Con contraseñas equivocadas, dejamos bloqueado al cadete de prueba del instituto durante 15 minutos. De ahí salió el hallazgo H5. También salió la regla de no correr nunca la suite en bucle contra el servidor, y de que las contraseñas vivan en secretos.
5. **Medir antes de optimizar.** Sin la línea de base del Sprint 3 contra la API real (0 de 30), "mejoramos la suite" no se podía demostrar.
6. **Hacer mejor no es hacer más.** Teníamos armados 33 casos y elegimos quedarnos con 22: la cobertura de riesgo es casi la misma y el mantenimiento es bastante menor.

8. Deuda técnica y próximos pasos

- **Corregir H5** (bloqueo de cuentas): es un defecto del sistema y hay que reportarlo al equipo de desarrollo. La suite puede sumar un caso de regresión cuando se corrija.
- Cubrir el rechazo de un viaje y su reasignación, y el no entregado: son los siguientes en riesgo.
- Validar el esquema completo de las respuestas exitosas, no solo de los errores (por ejemplo con `tv4` o `ajv` dentro de Postman).
- Cuando cambie la versión del backend del instituto, actualizar `BACKEND_REF` y correr la suite antes de desplegarlo: así la suite pasa a ser una puerta de calidad.

9. Distribución del trabajo

Las tareas se repartieron en cuatro partes de peso similar. Además, los cuatro revisamos la suite, los documentos y ensayamos la defensa en conjunto.

Integrante	Responsabilidad en el Sprint 4	Parte en la defensa
Leandro Raúl Vallejo	Organización del repositorio, documentación de la evolución de los sprints 1 a 3 y redacción del documento final.	Evolución del proyecto
Germán Garnica	Diseño y estabilización de los casos de prueba, flujo de un pedido y preparación de la demo en Postman.	La suite: casos, técnicas y demo
Jorge Alberto Medina	Mediciones (estabilidad, cobertura y tiempos), análisis de resultados y redacción de los hallazgos.	Hallazgos y métricas
Cesar Augusto Basaury Marquesto	Decisiones técnicas del sprint (estrategia de prueba, entorno aislado, qué automatizar y qué no), pipeline de CI/CD en GitHub Actions y publicación del reporte.	Pipeline CI/CD y decisiones técnicas